

Curpins

Curs 1

- Sisteme de calcul
 - Sisteme de operare (SO)
 - Servicii SO
-

Curs 2

- Bootarea sistemului
 - Kernelul
 - Runleveles
 - Logarea utilizatorului fara GUI
 - Interpretorul de comenzi (CLI)
 - Interfata grafica (GUI)
 - Identificare utilizatorului
 - Fisiere si directoare
-

Curs 3

- Mediu de lucru (environment)
 - Structura comenzilor in bash
 - Job control
 - Comenzi
 - Tipuri de fisiere
-

Curs 4

- Comenzi utile pentru fisiere
- Wild Cards
- Cautarea in fisiere
- Procese

- Comunicarea intre procese (IPC)
 - Metode de comunicare
 - Sincronizare
 - Semnale
-

Curs 5-6

- Variabile de mediu
 - Script
-

Curs 8

- Perspectiva utilizatorului
 - Perspectiva sistemului
 - Servicii
-

Curs 9

- Demoni
 - Transformarea programelor in demoni
 - Syslogd
 - Crond
 - Functia crond
 - Gestiunea timpului, ceasul HW
-

Curs 10

- Stocarea datelor
- Ierarhia in memorie
- Echipamente de tip bloc
- Echipamente loop
- Utilizarea echipamentelor bloc

- Fisiere
 - Sistemul de fisiere
-

Curs 11

- Retea
 - Protocol de comunicatie
 - Internet Protocol (IP)
 - TCP (Transport Control Protocol)
 - DNS (Domain Name System)
 - Socket
 - World Wide Web
 - Email IMAP, POP
 - Email SMTP
 - FTP (File Transfer Protocol)
 - Peer-to-peer
 - SSH (Secure SHell)
-

Curs 12

- de la sursa la executabil
- Loadtime
- Runtime
- Executabil simplu
- Obiecte
- Depanare (debugging)
- Proiecte

Curs 1

1. Sisteme de calcul

- Componente

- Hardware
- Sistem de operare
- Programe de aplicatie
- Utilizatori

- Operare

- procesele si controllere de echipamente comunica prin **magistrala** care asigura accesul la memorie => competitie pentru cicli de acces la memorie

2. Sistem de operare

- un program care intermediaza intre utilizator si HW

- obiective SO

- executa programele si usureaza solutionarea problemelor
- face sistemul de calcul usor de utilizat
- foloseste HW in mod eficient

- servicii SO

- **interfata utilizator** -> CL, GUI, touch-screen, Batch
- **executia programelor** -> sa incarce in memorie si sa-l execute, sa termine executia programului normal sau anormal(erori)
- **operatii de intrare/iesire**
- **manipularea fisierelor**
- **comunicatia** -> procesele pot schimba inf pe acelasi calc sau in retea
- **detectia erorilor**
- **alocarea resurselor**
- **protectie** (accesul la resurse) **si securitate** (fata de utilizatori externi)

Curs 2

1. Bootarea sistemului

- Se incarca un bootloader (GRUB) din ROM
- Kernelul este selectat si apoi incarcat si apoi sistemul ruleaza
- Loaderul identifica partitia de boot si incarca kernelul

- Kernelul

- initializeaza HW
 - Ramane prezent in memorie
 - Porneste primul proces: /sbin/init
- **init** => seteaza modul de operare (runlevel)

- Runlevels

- 0 , power-off
- 1, single-user mode
- 2, multi-user fara retea
- 3, multi-user cu retea dar fara interfata grafica
- 4, in general nedefinit, rezervat pt utilizari speciale
- 5, multi-user cu retea si interfata grafica
- 6, reboot

2. Logarea utilizatorului fara GUI

- **getty** => afiseaza prompt-ul de login
- **login** => afiseaza promptul de parola

3. Interpretorul de comenzi

- **Command Line Interpreter** (CLI), permite introducerea directa a comenzilor
- Asigura utilizatorului un mediu de lucru pentru:
 - Manipularea de fisiere; executia programelor; etc

4. Interfata grafica (GUI)

- Interfata user-friendly
 - compusa uzual din mouse, tastatura, si monitor
 - icoanele reprezinta fisierele, programele, actiuni, etc

5. Identificarea utilizatorului

- Userul primeste la login
 - un ID unic, UID = 0 fiind rezervat pentru `_root_` sau `_superuser_`
 - un GID (group id) care permite partajarea de resurse intre membrii aceluiasi grup,

6. Fisiere si directoare

- **Fisier** = abstractie la nivel de SO
 - Ascunde detaliile la nivel de HW pt stocarea efectiva a datelor
- **Director** = colectie de fisiere
 - Poate contine alte directoare
 - Stocheaza in mod ierarhic
 - Directoare speciale: ``.`` si ``.``
- Directorul in care se intaleaza discul formatat se numeste `_mouontpoint_`
- `ls -l <nume fisier>` => afisarea atributelor

Curs3

1. Mediu de lucru (environment)

- lista de perechi `name = value`
 - Variabile de mediu, ex: PS1, sunt accesibile din programe C din `char \*envp[]` setate cu comanda **export**
-

2. Structura comenzilor in bash

- **pipeline-uri** => executie paralela a comenzilor
\$ cmd1 | cmd2 | ... | cmdn
 - **liste de comenzi** => executie secventiala a comenzilor
\$ cmd1; cmd2; ...; cmdn
\$ cmd1 && cmd2 && ... && cmdn
\$ cmd1 || cmd2 ... || cmdn
-

3. Job control

- Programe rulate in foreground vs background ("&")
 - Comenzile din foreground se pot suspenda cu Ctrl+Z/C
-

4. Comenzi

- **Interne:** executate direct de catre bash
 - cd, exit, exec
 - **Externe:** programe de pe disc
 - pwd, ls, echo
-

5. Tipuri de fisiere

- **Fisiere obisnuite:** contin date
- **Directoare:** contin numele altor fisiere si informatii despre ele
 - Pot fi citite de procesele care au permisiunile potrivite
 - DOAR kernelul poate scrie in ele
- Fisiere speciale, tip device
 - Character

- [[Cursul 10#Echipamente de tip bloc|Bloc]]
- Orice device din sistem e fie fisier bloc, fie caracter
- **FIFO**: pipe, Inter-Process-Communication (IPC) => **conecteaza procese fara leg**
- **Socket**: abstractie pentru IPC peste retea
- **Link simbolic** (shortcut): fisier care contine numele fisierului referit (un string)
 - Hard: link-ul fisierului linkat trebuie sa se afle pe acelasi sistem de fisiere
- Fiecare proces are asociat un UID/GID

Curs 4

1. Comenzi utile pt fisiere

- mkdir => creeaza directoare
 - rmdir => sterge directoare (goale)
 - touch => creaza un fisier gol daca nu exista deja
 - mv => muta directoare sau fisiere
-

2. Wildcards

- ^ => inceputul liniei
 - \$ => sfarsitul liniei
-

3. Cautarea in fisiere

- grep \$ grep <expresie> <fisiere>
-

4. Procese

Abstractia executiei unui program

- Compuse din instructiuni si date
- Identificat prin Process ID (PID)
- Mai multe instante in rulare ale aceluasi program sunt procese diferite, cu PID-uri diferite
- Executia lor este **secventiala**, nu exista executie paralela a instructiunilor intr-un singur proces
- Format din:
 - Codul program (sectiunea .text)
 - Starea curenta (registrele CPU)
 - Stiva (date temporare)
 - Sectiunile .data
 - Heap
- comanda **ps** afiseaza PID-urilor proceselor aflate in rulare momentan

5. Comunicare intre procese (IPC)

- Procesele pot fi independente sau cooperante
- Procesele cooperante pot afecta sau pot fi afectate de alte procese, inclusiv prin partajarea datelor

6. Modele de comunicare

- Memorie partajata
- Schimb de mesaje
 - Operatii: send(), receive()
 - **Directa**: procesele isi folosesc identitatea explicit
 - send(P, message)
 - receive(Q, message)
 - ex: sockets
 - **Indirecta**: mesajele sunt pasate prin casute postale (**porturi**)
 - send(A, message)
 - receive(A, message)

7. Sincronizare

Schimbul de mesaje poate fi

- **blocant** (sincron)
 - send blocant – transmitatorul e blocat pana cand se primeste mesajul
 - receive blocant – receptorul e blocat pana cand un mesaj e disponibil
- **neblocant** (asincron)
 - send neblocant – transmitatorul trimite mesajul si continua
 - receive neblocant – receptorul primeste:
- Daca send si receive sunt ambele blocante, avem un ****rendezvous****

8. Semnale

- Notificari asincrone
- Echivalentul software al exceptiilor (HW sau SW)
- Se trimit fie intre procese, fie de catre kernel catre un proces

Curs 5-6

1. Variabile de mediu

- Folosite de sistem pentru a defini modul de functionare a programelor
 - Variabile importante, ex:
 - \$HOME, \$PATH, \$SHELL, \$EDITOR, \$LANG
-

2. Script

- Program scris pentru un mediu run-time specific care automatizeaza executia comenzilor ce ar putea fi executate alternativ manual de catre un operator uman
- Nu necesita compilare
- Executia se face direct din codul sursa
- Programele ce executa scriptul se numesc interpretoare in loc de compilatoare
- Ex: perl, ruby, python, bash

Curs 8

1. Perspectiva utilizatorului

- Accesul utilizatorului in sistem
- Interpretorul de comenzi

2. Perspectiva sistemului

- Serviciile SO accesibile in doua moduri:
 - Direct: prin **kernel**
 - Indirect: prin programe din userspace (care tot apeleaza kernelul)
- **Comenzile shell:**
 - Apeleaza direct serviciile kernelului *_sau_*
 - Contacteaza servere din userspace pentru efectuarea serviciului
- **Servere (aka demoni)**
 - Programe specializate care furnizeaza servicii ale SO in spatiul utilizatorului
 - Accesibile prin diferite moduri de IPC
 - ex: servicii de retea, firewall, imprimare, gestiune a timpului, securitate, etc

3. Servicii

- Programe de sistem pornite la bootarea SO
- Fie **servere/demoni**, fie programe care contribuie la buna functionare si la asigurarea mediului de executie pentru programele utilizatorului
- Procese pornite de **`init`**
- Ruleaza atata vreme cat sistemul este in functiune
- Configurate pentru fiecare runlevel in parte
- Multiple interfete de acces si gestiune:
 - System V (Unix)
 - Upstart (implementare Linux pt **`init`**)
 - Systemd (varianta recenta Linux)

Curs 9

1. Demoni

- Procese (servere) care ruleaza in background si nu sunt asociate cu un terminal de control
 - In general porniti ca servicii de sistem la sfarsitul operatiei de boot
-

2. Transformarea programelor in demoni

- 1) Detasarea de terminal
 - 2) Schimbarea directorului de lucru curent
 - 3) Inchiderea tuturor descriptorilor de fisiere deschisi
 - 4) Redirectarea `_stdin`, `stdout` si `_stderr` la `/dev/null`
 - 5) Utilizarea `_syslogd` pentru logarea erorilor
-

3. Syslogd

- Fara terminal demonii necesita metode specifice pentru a afisa mesaje
 - Metoda standard foloseste functia `_syslog` care trimite mesaje catre demonul `_syslogd` (Echivalentul de Event Viewer din Windows)
 - Functia `_syslog` si `_logger` permit scrierea mesajelor in log-uri
 - Specifica tipul de program care trimite mesajul si valoarea
-

4. Crond

- Demon care executa comenzi planificate pentru rulare la un moment dat (uzual periodic)
- Pornit ca serviciu de bootare
- Incarca in memorie tabele cu planificarile taskurilor utilizatorului, `_crontabs`

5. Functionarea Crond

- La fiecare 60s examnizeaza continutul crontaburilor pentru a stabili daca respectivele comenzi trebuie rulate in timpul minutului curent; se verifica si daca directorul de spool sau tabela de sistem s-au modificat

6. Gestiunea timpului, ceasul HW

- Doua ceasuri: HW si sistem
- Ceasul sistem:
 - Componenta a kernelului bazata pe un timer (intrerupere de timp)
 - Are sens doar cat merge calculatorul
 - Initializat din ceasul HW

Curs 10

1. Stocarea datelor

- Memoria principala
 - Singurul mediu de stocare de dimensiune mare accesibil direct procesorului
 - Acces random
 - Uzual volatilă
 - Tipic DRAM
- Stocare secundară
 - Extensie a memoriei principale
 - Nevolatilă
 - ex: Hard disk, SSD, DVD, etc

2. Ierarhia de memorie

- Sistemele de stocare organizate în ierarhii în funcție de viteză, cost, volatilitate
- **Caching:**
 - Copiază informația în sisteme de stocare mai rapide
- **Driverul de echipament** (Device Driver)
 - Operează fiecare device controller pentru a gestiona operațiile de I/O

3. Echipamente de tip bloc

- Unitatea de transfer a datelor = blocul de date
- Arhitecturile de calcul moderne bazate pe Direct Memory Access (DMA)
 - Transfer de date între RAM și echipamentul bloc neintermediat de CPU
- Reprezentate în sistemele Unix ca fișiere speciale de tip bloc în directorul `/dev`
- Caracterizate de:
 - ``nr major``: identifică driverul asociat echipamentului
 - ``nr minor``: nr unități de acel tip din sistem

4. Echipamentele loop (loop devices)

- Pseudo-echipament care permite folosirea unui fisier obisnuit ca echipament bloc

5. Utilizarea echipamentelor bloc

- Stocarea datelor
 - Datele inregistrate pe discuri conform unui format specific unui anumit tip de sistem de fisiere (ext4, ntfs, etc)
 - Inregistrarea formatului sistemului de fisiere pe disc ("formatarea discului")
- Spatiu de swap
 - Discuri neformatate
 - Memorie virtuala, folosita drept RAM pentru programe mai mari

6. Fisiere

- Abstractie la nivel de SO pentru stocarea datelor
- Low level, stocarea persistenta se face pe discuri
- Folosite uzual prin operatii de `open-read/write-close`
- Pe langa date, stocheaza si **metadate**
 - Exemple: data ultimului acces/modificari, proprietarul fisierului, permisiuni, dimensiune, etc
 - Structura de acces la reprezentarea low-level a datelor (adrese de blocuri pe disc)

7. Sistemul de fisiere

- Componenta speciala a SO care gestioneaza fisierele si directoarele
- Structureaza datele pe disc intr-un anumit `_format_` (ex: ext4, ntfs)
- Oferă utilizatorului o interfata uniforma la acces de date

- Tipuri:
 - Pentru stocare permanenta
 - Sisteme temporare
 - Sisteme bazate pe echipamente loop
 - Sisteme de fisiere distribuite
 - Pseudo-sisteme de fisiere

Curs 11

1. Retea

- Alcatuit din noduri (calculatoare) numite host-uri
- Legaturi fizice: cabluri de retea, canale wireless
- Legaturi logice: calea prin intermediul legaturilor fizice dintr-un nod sursa si unul destinatie
- O retea poate fi conectata la una sau mai multe retele

2. Protocol de comunicatie

- Modelul OSI (Open Systems Interconnection)
 - 7 Nivele: fizic, data link, retea, transport, sesiune, prezentare, aplicatie
- Model alternativ, protocoalele internet
 - Combina utlimele trei nivele OSI (sesiune, prezentare, aplicatie) intr-un singur nivel: *_aplicatie_*
 - Combina nivelul fizic si data link intr-un singur nivel: *_link_*
- Functioneaza pe principiul stivei de protocoale:
 - Protocolul de nivel cel mai inalt apeleaza la serviciile protocolului imediat anterior
 - La baza, datele trimise prin protocoalele *_data link_* sunt trimise prin retea fizica
- Exemple:
 - Data link (retele LAN): Ethernet, Wireless
 - Retea: IP
 - Transport de date: TCP, UDP, SCTP
 - Sesiune: RPC
 - Prezentarea datelor: criptare, compresie, XDR, ASN.1
 - Aplicatie: HTTP, DNS, FTP, SSH
- Socket: abstractizeaza comunicatia la nivel de aplicatie

3. Internet Protocol (IP)

- Fiecare host este identificat prin una sau mai multe adrese IP
- Versiuni: IPv4 (32 biti) si IPv6 (128 biti)
- Anumite adrese sunt folosite pentru adresare privata
 - Ex: 192.168.1.1, 127.0.0.1

4. TCP (Transport Control Protocol)

- Impacheteaza/dezpacketeaza mesaje in pachete
- Pe scurt, este cel mai safe protocol:
- Asigura livrarea pachetelor in ordine la destinatie
- Rezolva automat probleme de rutare
- Asigura retransmisia automata a pachetelor pierdute/defecte

5. DNS (Domain Name System)

- Inlocuieste IP-uri cu nume usor de tinut minte
- Agenda centralizata de perechi (nume, ip)
- Ex: fmi.unibuc.ro -> 193.226.51.6

6. Socket

- Abstractie de nivel SO
- Defineste un endpoint format din:
 - Adresa: adresa IP a unui host
 - Port: intrarea a host-ului
 - Port server: identifica un serviciu
 - Port client: identifica procesul client
- O conexiune se face intre 2 socketuri/endpointuri folosind un protocol comun
- Ex: <TCP, 192.168.1.6:4444, 193.226.51.6:80>

- TCP Sockets:
 - Beneficiaza de safety-ul protocolului [[#TCP (Transport Control Protocol)|TCP]]
 - Din el se citesc octeti, nu mesaje intregi
- UDP Sockets:
 - Canal *_fara garantii_* (fara safety)
 - Mesajele se pot pierde
 - Mesajele se citesc in intregime

7. World Wide Web (www)

- Foloseste HyperText Transfer Protocol (HTTP) si portul 80
- Resursele sunt identificate prin URL
- Acces criptat prin HTTPS pe portul 443
- Exemplu comincatie HTTP:

...

GET/HTTP/1.1

Host: <host>

...

8. Email - IMAP, POP

- Acces la posta
- Post Office Protocol (POP)
 - Face o copie locala a mesajelor si le sterge de pe serverul de mail
- Internet Message Access Protocol (IMAP)
 - Opereaza asupra mesajelor direct pe server
 - Poate tine o copie (partiala) local

9. Email - SMTP

- Expediere mesaje
- Simple Message Transfer Protocol (SMTP)
 - Trimite un mesaj catre un destinatar
 - Este un protocol vechi, nu cere expeditor si nici autentificare care duce la spam si spoofing
- Solutii anti-spam:
 - Reverse DNS: legatura IP -> nume
 - Autentificare: utilizator si parola; criptare
 - Anti-email spoofing (SPF, DKIM, DMARC)
 - Baze de date cu IP-uri de spammeri

10. FTP (File Transfer Protocol)

- Transfer de fisiere
- Autentificare si criptare FTPS

11. Peer-to-peer

- Transfer de fisiere distribuit fara server
- Toti clientii transmit si primesc date
- Mod eficient de transfer pentru date mari
- Ex: Torrent

12. SSH (Secure SHell)

- Protocol sigur de conectare la alt calculator
- Folosit peste tot in programare si administrare
- Autentificare parola, cheie asimetrica, etc

Curs 12

1. De la sursa la executabil

- Sursa - fisierul scris in limbaj de programare
 - Compilator - traduce sursa in limbaj masina
 - Obiect - fisier binar; rezultat al compilarii
 - Biblioteca - fisier binar ce ofera o anumita functionalitate
 - Linker - leaga obiecte si biblioteci pentru a produce executabilul
 - Executabil
 - Static: toate obiectele si bibliotecile sunt incluse in fisier
 - Dinamic: contine doar datele si instructiunile proprii plus apeluri catre biblioteci; bibliotecile sunt pastrate in fisiere separate
-

2. Load time

- Rezolvarea dependentelor executabilelor dinamice
 - 1) SO incarca executabilul in memorie
 - 2) Se cauta bibliotecile folosite (in caz ca nu se gasesc, se opreste executia)
 - 3) Se incarca bibliotecile necesare
-

3. Run time

- Rezolvarea dependentelor executabilelor dinamice la nevoie
 - 1) SO incarca executabilul in memorie
 - 2) Se cauta bibliotecile *strict necesare lansarii* programului
 - 3) Pe parcursul executiei daca este nevoie de o functie de biblioteca
 - a) Executia se opreste
 - b) Se cauta biblioteca (in caz ca nu se gasesc, se opreste executia)
 - c) Se incarca biblioteca
 - d) Se rezuma executia

4. Executabil simplu

- In forma cea mai simpla, compilatorul:
 - Primeste ca argumente fisierele sursa
 - Produce executabilul

4. Obiecte

- Compilatorul se opreste dupa producerea sa
- Nu continua cu crearea unui executabil
- Obiectul va fi folosit impreuna cu alte obiecte si biblioteci pentru a produce executabilul final

6. Depanare (debugging)

- Programul cel mai comun: gdb

- Break/breakpoint
 - Punct in care sa se opreasca executia
- Backtrace/trace/stack trace
 - Apelul de functii care a dus in punctul curent
- Watch/watchpoint
 - Punct in care sa se opreasca executia daca este indeplinita o conditie

7. Proiecte

- Produsele software sunt alcatuite din mai multe componente:
- Module
- Biblioteci
- Fisiere sursa, intre care apar dependente
- Dependentele si ordinea de compilare se pot rezolva printr-un Makefile